Attention Is All You Need

A structured guide to the original Transformer paper and its core ideas

How to use these notes

Read once for the big picture, then revisit the definitions, examples, and comparison tables. The aim is to build intuition first and connect the ideas to practical AI systems.

1. The problem before Transformers

Earlier sequence models often used recurrent neural networks (RNNs), including LSTMs and GRUs, to process tokens one at a time. This sequential processing made it harder to parallelize training and could make long-range relationships difficult to learn.

The 2017 paper *Attention Is All You Need* proposed the Transformer: an architecture built around attention rather than recurrence or convolution. It was introduced for machine translation, but its design later became foundational for many language models.

Core idea: let each token directly gather information from other relevant tokens, instead of relying only on a sequential hidden state.

2. What is attention?

Attention computes how much one token should use information from other tokens. In self-attention, queries, keys, and values are produced from the same input sequence.

Queries, keys, and values

- **Query (Q):** what this token is looking for.
- **Key (K):** what each token offers for matching.
- **Value (V):** the information that can be collected if a token is attended to.

The model compares a query with keys to produce attention scores. A softmax turns the scores into weights, and the weights combine the value vectors.

Scaled dot-product attention

The paper defines attention as:

```
Attention(Q, K, V) = softmax(QK■ / √d■) V
```

The division by √d■ helps keep dot-product scores from becoming too large as the key dimension grows, which can make softmax gradients less useful.

3. Self-attention, step by step

- Start with token representations for a sentence.
- Project each representation into Q, K, and V vectors using learned weight matrices.
- Calculate query–key similarity scores across token positions.

- Scale the scores and apply softmax to get attention weights.
- Use the weights to form a weighted sum of value vectors.
- Pass the updated representations to later layers.

Example: in “The animal didn’t cross the street because it was tired,” attention can help the representation of “it” draw useful information from “animal.” The model learns these relationships from data; it is not given grammar rules by hand.

4. Multi-head attention

Instead of using a single attention operation, the Transformer uses multiple attention heads in parallel. Each head has its own learned projections and can learn different relationships.

- One head might focus on nearby words.
- Another might connect a pronoun to a noun.
- Another might track syntactic or semantic relationships.

The outputs of the heads are concatenated and projected to produce the multi-head attention output. These examples describe possible behavior, not fixed roles assigned to heads.

5. Positional information

Self-attention by itself does not inherently encode token order. The original Transformer adds positional encodings to token embeddings so the model can distinguish different positions.

The paper used sine and cosine functions at different frequencies. These encodings provide position-dependent signals, allowing the model to reason about token order and relative offsets.

Token embeddings answer “what token is this?” Positional information helps answer “where is it in the sequence?”

6. Encoder and decoder architecture

Encoder

The original encoder stacks layers containing multi-head self-attention and a position-wise feed-forward network. Each sublayer uses a residual connection and layer normalization.

Decoder

The decoder uses masked self-attention so a position cannot look at future output tokens during generation. It also uses encoder–decoder attention to access the source sequence, followed by a feed-forward network.

This encoder–decoder structure was designed for sequence-to-sequence tasks such as translating a source sentence into a target sentence.

7. Feed-forward layers, residuals, and normalization

Attention mixes information across token positions. A position-wise feed-forward network then applies learned transformations to each position independently, using the same parameters at every position.

Residual connections provide a shortcut around a sublayer, helping information and gradients flow. Layer normalization stabilizes the representation scale. The paper’s exact arrangement is part of the original architecture; modern Transformer variants may differ.

8. Training objective and inference

For translation, the decoder is trained to predict the next target token while conditioning on the source sentence and previous target tokens. During training, the correct previous tokens are commonly supplied (teacher forcing); at inference, generated tokens are fed back step by step.

The paper used a large parallel translation dataset and label smoothing. Its results showed that a Transformer could achieve strong translation quality with substantially more parallelizable training than recurrent architectures.

A model's training objective is not the same as its deployment behavior. Modern large language models often use Transformer-based architectures but may be decoder-only and trained with next-token prediction rather than the original encoder–decoder translation setup.

9. Why the paper mattered

- Parallelism: token positions can be processed in parallel within a layer during training.
- Long-range connections: attention can connect distant tokens directly.
- Flexible context use: different heads can learn different patterns.
- Scalable foundation: the architecture became central to many translation, language, vision, and multimodal systems.

Trade-off: standard full self-attention compares all pairs of positions, so its attention computation and memory use typically grow quadratically with sequence length. Long-context systems use optimizations or alternative attention patterns to reduce this cost.

10. Key equations and vocabulary

Useful terms

- **Token:** a unit of text processed by the model, often a word piece rather than a full word.
- **Embedding:** a learned vector representation of a token.
- **Attention weight:** how strongly a query uses a particular value position in that operation.
- **Logits:** unnormalized scores for possible next tokens.
- **Softmax:** converts scores into normalized non-negative weights or probabilities.
- **Autoregressive generation:** producing one token at a time while conditioning on prior tokens.

Remember the pipeline: token IDs → embeddings + positions → attention and feed-forward layers → output scores → next-token prediction.

11. Common misconceptions

- Attention is not a database lookup. It calculates learned, input-dependent weighted combinations inside a neural network.
- An attention weight is not a guaranteed explanation of the model's reasoning.
- The original paper did not introduce every modern LLM design. It introduced the Transformer architecture in an encoder–decoder translation setting.
- A Transformer is not automatically an LLM. The architecture, training data, objective, scale, and fine-tuning all matter.

12. Check your understanding

Questions to answer without looking

- Why can self-attention connect distant tokens more directly than an RNN?
- What is the difference between a query, key, and value?
- Why divide attention scores by √d■?
- Why does the decoder mask future positions?
- What problem does positional encoding solve?
- What is the main computational trade-off of full self-attention?

Suggested reading

Vaswani et al. (2017), *Attention Is All You Need*. Read the abstract, Section 3 on model architecture, and the attention equation first. Then review the training and results sections for the original translation context.